PT100 Digital Thermometer

Project Progress Log

Arduino Measurement, Least-Squares Calibration, LCD Integration and Enclosure Development

Yashraj Kranthi

16 October 2026

Abstract

This report documents the development of an Arduino-based digital thermometer using a PT100 resistance temperature detector (RTD). The work covers the measurement circuit, collection and processing of analogue readings, least-squares calibration, Arduino software, a JHD162A 16-by-2 character LCD, Celsius/Fahrenheit selection, troubleshooting, and mechanical enclosure design.

The Arduino program calculates mean readings from analogue inputs A0 and A1, forms a differential feature, estimates temperature using fitted calibration coefficients, and prints intermediate values to the Serial Monitor. During testing, a maximum observed error of approximately 0.7 degrees Celsius was reported. This is a preliminary observed result rather than a guaranteed accuracy specification: a final claim requires documented comparisons with a suitable reference thermometer across the intended operating range.

The report separates observations already recorded from features that still require final verification. Photographs can be inserted by replacing the image placeholder filenames with the corresponding files from the project's thermometer repository.

Contents

1 Project Aim and Requirements **4**

2 Hardware and Measurement Principle **5**

2.1 PT100 resistance-temperature relationship 5

2.2 Analogue measurements 5

2.3 Initial hardware checks 6

2.4 Measurement-chain limitations 6

3 Calibration Data and Least-Squares Fit **7**

3.1 Reason for calibration 7

3.2 Linear least-squares model 7
3.3 Calibration workflow 7
3.4 Residuals and evaluation metrics 8
3.5 Calibration range and extrapolation 8

4 Arduino Software and Diagnostic Output 9
4.1 Software responsibilities 9
4.2 Recorded serial-monitor example 9
4.3 Celsius and Fahrenheit conversion 10
4.4 D8 push-button behaviour 10
4.5 LCD formatting 10

5 LCD Integration and Troubleshooting 11
5.1 Reported display issue 11
5.2 Troubleshooting sequence 11
5.3 Separating measurement faults from display faults 12

6 Observed Accuracy and Validation Plan 12
6.1 Reported maximum error 12
6.2 Recommended validation procedure 12

7 Enclosure Design and Mechanical Integration 14
7.1 Design objective 14
7.2 Recorded dimensions 14
7.3 Raised LCD support platform 15
7.4 Captive push-button plunger 15
7.5 Potentiometer and PT100 lead opening 16
7.6 Internal mounting and protection 16
7.7 Printability and structural checks 16
7.8 Ventilation and sensor placement 17

8 System-Level Testing Checklist 17

9 Challenges and Lessons Learned 18
9.1 A plausible result may still be outside calibration 18
9.2 Debug the signal chain in stages 18

9.3 Accuracy and display resolution are different 18
9.4 Mechanical design affects electrical reliability 18

10 Conclusion and Remaining Work **18**

1 Project Aim and Requirements

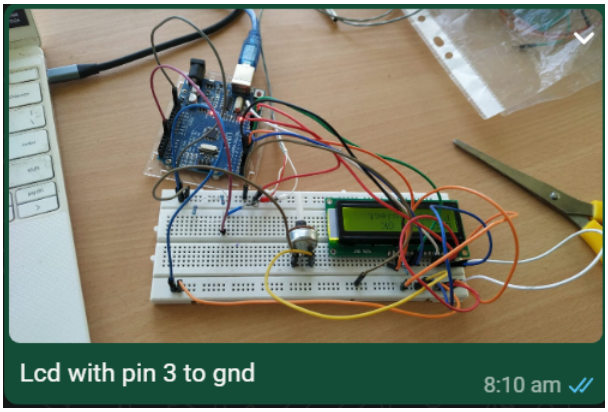
The aim was to build a functional digital thermometer around a PT100 sensor and an Arduino Uno R3. The sensor's electrical behaviour is measured by an analogue circuit, the Arduino converts the readings into a temperature estimate, and an LCD presents the result to the user.

The project involved more than displaying a numerical value. The instrument also needed a calibration method, diagnostic output for troubleshooting, a clear indication when a reading fell outside the supported calibration range, and a protective enclosure for the assembled electronics.

The principal objectives were:

1. Interface the existing PT100 measurement circuit with the Arduino.
2. Collect calibration data and fit a least-squares model.
3. Implement the model in Arduino C++.
4. Display the temperature and measurement status on a JHD162A-compatible 16-by-2 LCD.
5. Use a push button connected to digital pin D8 to toggle between Celsius and Fahrenheit.
6. Diagnose incorrect or incomplete LCD output and investigate calibration-status messages.
7. Evaluate the observed temperature error and document the limitations of the available tests.
8. Design an enclosure that accommodates the Arduino, perfboard, LCD, controls, wiring and PT100 lead.

The development was performed around the existing hardware. The intention was to improve and validate the assembled prototype rather than replace it with an unrelated design.



2 Hardware and Measurement Principle

2.1 PT100 resistance-temperature relationship

A PT100 is a resistance temperature detector with a nominal resistance of 100 ohms at 0°C. Its resistance changes with temperature. For temperatures at or above 0°C, the standard platinum RTD relationship can be approximated by

$$R(T) = R_0(1 + AT + BT^2), \qquad T \geq 0^\circ\mathrm{C},$$

where $R(T)$ is the resistance at temperature T, R_0 is the nominal resistance at 0°C, and A and B are coefficients defined by the relevant RTD standard. For a nominal IEC 60751 platinum RTD, commonly used coefficients are approximately $A = 3.9083 \times 10^{-3}\,\mathrm{degC}^{-1}$ and $B = -5.775 \times 10^{-7}\,\mathrm{degC}^{-2}$. The exact sensor specification should be checked before using these coefficients as a reference model.

The Arduino does not necessarily measure PT100 resistance directly. Instead, the measurement circuit converts the sensor behaviour into voltages that can be sampled by the analogue-to-digital converter (ADC). The calibration model then maps measured circuit behaviour to temperature.

2.2 Analogue measurements

The Arduino Uno R3 has a 10-bit ADC, giving nominal digital readings from 0 to 1023. With the conventional reference configuration, the input voltage can be approximated by

$$V_{\mathrm{in}} \approx \frac{N_{\mathrm{ADC}}}{1023} V_{\mathrm{ref}},$$

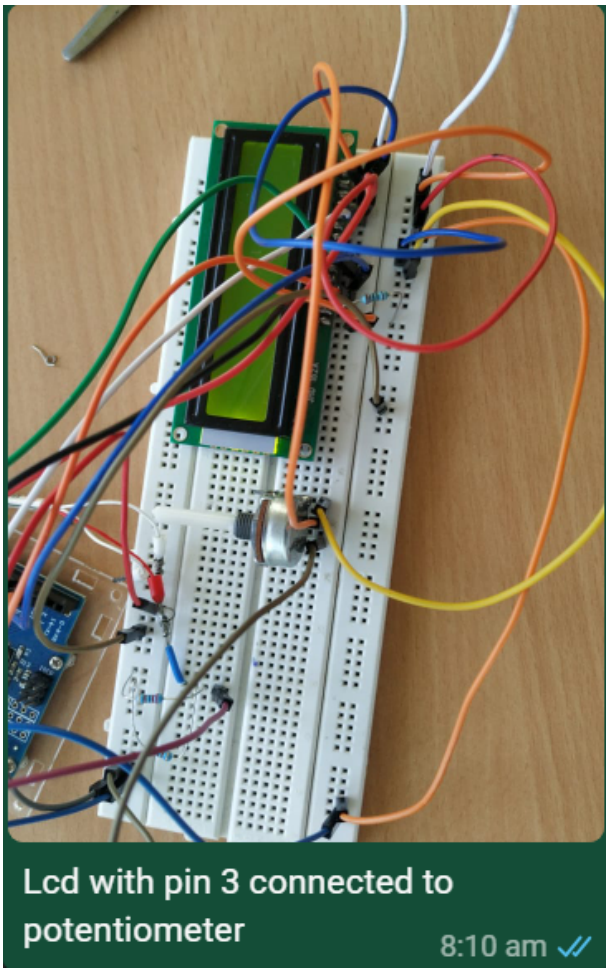
where N_{ADC} is the ADC count and V_{ref} is the actual ADC reference voltage. Reference-voltage variation and analogue-circuit tolerances affect the relationship between counts and voltage.

In the recorded software workflow, repeated readings from A0 and A1 were averaged. The means were combined into one feature:

$$\bar{N}_{A0} = \frac{1}{n}\sum_{i=1}^{n} N_{A0,i}, \qquad \bar{N}_{A1} = \frac{1}{n}\sum_{i=1}^{n} N_{A1,i},$$

$$x = \bar{N}_{A0} - \bar{N}_{A1}.$$

Averaging helps reduce uncorrelated random noise. It does not, by itself, remove systematic errors caused by reference-voltage variation, component tolerances, lead resistance, poor connections or sensor self-heating.



2.3 Initial hardware checks

The basic checks during assembly and integration were to verify that the Arduino powered correctly, analogue inputs returned changing values, the LCD had power and contrast adjustment, and the wiring was consistent with the code. The push-button input also needed an electrical configuration that matched the software logic.

A stable serial reading is not proof that the temperature is correct, but observing the raw or averaged ADC values is a useful first diagnostic. If the input values do not respond to changes at the sensor, the measurement hardware should be checked before modifying the temperature-calibration calculations.

2.4 Measurement-chain limitations

The sensor signal depends on the circuit used to excite and measure the PT100. Excitation current, resistor tolerances, voltage-reference stability, wiring resistance and electrical noise can all affect the result. Sensor self-heating is another possible source of error if too much current passes through the element.

Calibration can compensate for repeatable behaviour within the observed calibration dataset. It cannot guarantee compensation for changes in wiring, hardware, ambient

conditions or operation outside that dataset.

3 Calibration Data and Least-Squares Fit

3.1 Reason for calibration

Raw ADC counts do not have units of degrees Celsius. A calibration procedure is therefore required to translate the analogue feature into a temperature estimate. The project used collected calibration data to fit the relationship for the assembled circuit.

A data-driven fit can accommodate repeatable behaviour of the complete measurement chain, but its validity depends on the quality and range of the calibration readings. Reference temperatures must be recorded carefully, and the sensor and reference thermometer should be allowed to reach thermal equilibrium before data are collected.

3.2 Linear least-squares model

For a linear model, each calibration point is represented by

$$T_i = \beta_0 + \beta_1 x_i + \epsilon_i,$$

where T_i is the reference temperature, x_i is the measured feature, β_0 is the intercept, β_1 is the slope, and ϵ_i represents measurement noise and model mismatch.

The least-squares estimate chooses coefficients that minimize the sum of squared residuals:

$$S(\beta_0, \beta_1) = \sum_{i=1}^{n} \left[T_i - (\beta_0 + \beta_1 x_i)\right]^2 .$$

In matrix form, the coefficient estimate is written as

$$\hat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \mathbf{T},$$

provided the inverse exists. This equation describes the ordinary least-squares solution; software implementations can use numerically stable methods such as QR decomposition rather than explicitly calculating the inverse.

The fitted coefficients are then used in the Arduino program to estimate temperature from a new measurement feature.

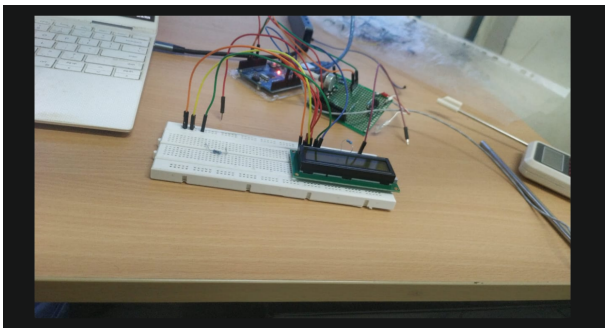
3.3 Calibration workflow

The workflow followed by the project was:

1. Collect measurement readings at known or reference temperatures.
2. Average repeated analogue samples where appropriate.

3. Calculate the feature $x = \bar{N}_{A0} - \bar{N}_{A1}$ for each calibration point.
4. Fit the selected model to the feature and reference-temperature data.
5. Transfer the fitted coefficients into the Arduino calibration code or its configuration.
6. Inspect the residuals and check the model against the collected data.
7. Test additional readings and determine whether they fall inside the supported calibration domain.

The fitting dataset and independent validation dataset should be distinguished. A low error on the same data used to fit the model can overstate performance on new measurements.



3.4 Residuals and evaluation metrics

For each test point, the residual can be defined as

$$e_i = T_{i,\text{reference}} - T_{i,\text{estimate}}.$$

Useful error summaries include mean absolute error (MAE), root mean squared error (RMSE), and maximum absolute error:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|,$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2}, \qquad e_{\max} = \max_i |e_i|.$$

These statistics are meaningful only when the temperatures, reference values, and conditions under which the readings were recorded are documented. Results from calibration points should not be presented as independent validation results.

3.5 Calibration range and extrapolation

A fitted model is most defensible within the region covered by its calibration data. Outside this region, the model may still return a number, but that result is an extrapolated estimate. The software therefore reports a calibration status, including the status string `OUT_OF_CAL_ESTIMATE` in the recorded serial output.

This status is important: a plausible numerical value does not by itself establish that the estimate is validated for that input. The program's calibration limits should be checked against the actual minimum and maximum feature values in the accepted calibration dataset.

4 Arduino Software and Diagnostic Output

4.1 Software responsibilities

The Arduino code was developed to perform the following tasks:

- acquire measurements from analogue inputs A0 and A1;
- average repeated ADC samples;
- calculate the differential feature;
- evaluate the calibration model;
- determine whether the feature lies within the calibration domain;
- format temperature and status information for the LCD;
- print intermediate values to the Serial Monitor;
- support Celsius/Fahrenheit selection using a button on D8.

Separating acquisition, calibration and presentation makes faults easier to locate. A formatting problem in the LCD code should not be mistaken for an error in the fitted model, and a plausible serial value does not prove that the calibration limits are correct.

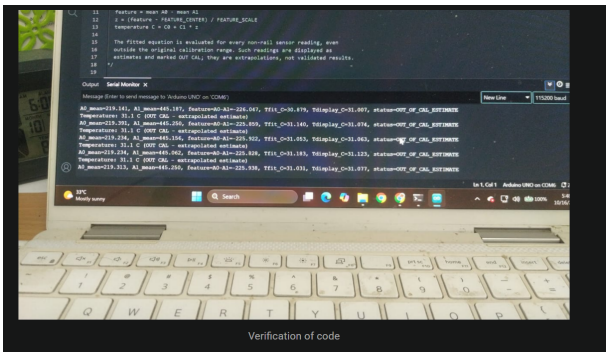
4.2 Recorded serial-monitor example

One recorded diagnostic output was:

```
A0_mean=218.875, A1_mean=444.781,
feature=-225.906, Tfit_C=31.075,
Tdisplay_C=31.050,
status=OUT_OF_CAL_ESTIMATE
```

For this sample, the program reported a fitted estimate of approximately 31.075°Cand a display variable of approximately 31.050°C. The difference between those values is small, but it should be accounted for by the program's rounding, filtering or other explicitly implemented processing.

The status was `OUT_OF_CAL_ESTIMATE`, so this example should be treated as an out-of-domain estimate rather than a fully validated measurement. The negative feature is not automatically evidence of faulty hardware: the sign depends on the order of subtraction and on the responses of A0 and A1. It should, however, agree with the feature values and coefficient convention used during calibration.



4.3 Celsius and Fahrenheit conversion

The Celsius-to-Fahrenheit conversion is

$$T_F = \frac{9}{5}T_C + 32,$$

and the inverse conversion is

$$T_C = \frac{5}{9}(T_F - 32).$$

The calibration calculation should remain in Celsius when its coefficients were fitted to Celsius reference values. Fahrenheit should be calculated from the Celsius estimate for display. This avoids maintaining two separate sets of calibration coefficients and reduces the risk of mixing units.

4.4 D8 push-button behaviour

A push button connected to digital pin D8 was specified to toggle the displayed unit. The intended sequence is to start in Celsius mode, switch to Fahrenheit after one press, and return to Celsius after the next press.

The code should respond once to a new press rather than toggling continuously while the button remains held. Mechanical contacts can bounce, so debouncing or a state-transition method is needed. The button's physical wiring must also match the input logic. With `INPUT_PULLUP`, a common arrangement connects the switch between D8 and ground, so the pressed state reads `LOW`. An external pull-down arrangement typically uses the opposite logic.

The feature should be considered verified only after confirming both unit transitions on the physical prototype and checking that a held button does not cause repeated changes.

4.5 LCD formatting

A 16-by-2 LCD has limited room. A suitable compact layout might display temperature on the first line and the calibration state on the second. For example:

```
Temp: 31.1 C
CAL: IN RANGE
```

For an out-of-domain estimate, the status might instead be shortened to `OUT OF CAL`. These are example layouts, not a claim that the final display has already been verified. Each line must fit the available 16 character positions, and unused characters from a previous, longer string should be overwritten to prevent remnants from remaining on screen.

5 LCD Integration and Troubleshooting

5.1 Reported display issue

During integration, the LCD was reported to show text resembling:

```
temp: ?C OUT CAL D: ?
```

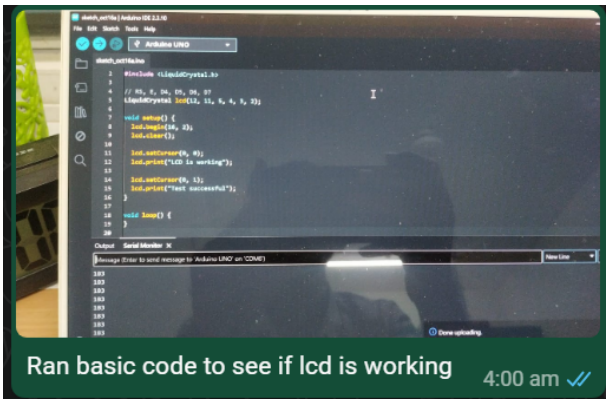
This output did not present the temperature as intended. A question mark in place of a digit can result from the value passed to the display routine, string formatting, data-type handling, unexpected characters or an LCD update issue. The text alone is not enough to identify a single cause.

5.2 Troubleshooting sequence

The appropriate checks were to:

1. verify the LCD supply and shared ground;
2. compare the actual wiring with the pin mapping in the program;
3. adjust the contrast potentiometer;
4. run a fixed-message LCD test before adding sensor calculations;
5. print the computed temperature to the Serial Monitor independently;
6. check variable types and formatting where the temperature string is constructed;
7. verify the number and order of arguments passed to LCD functions;
8. make sure both LCD lines are updated without leaving stale characters.

If a fixed message displays correctly while the temperature does not, attention should shift to the measurement value and formatting code. If no characters display, the power, contrast, communication wiring and initialization should be checked first.



5.3 Separating measurement faults from display faults

Serial diagnostics allow the measurement path to be tested without relying on the LCD. If the serial output shows a plausible estimate but the display is incorrect, the LCD formatting and update logic should be investigated separately. If the ADC readings themselves are unstable or implausible, the analogue circuit and wiring need attention first.

The calibration-status result should be examined independently as well. An estimate can look plausible while still being outside the calibrated feature range.

6 Observed Accuracy and Validation Plan

6.1 Reported maximum error

A maximum observed error of approximately 0.7°Cwas reported during testing. This is a preliminary result from the tests performed; it is not yet a certified accuracy specification. The interpretation depends on the reference instrument, the number of test points, whether the sensor had reached thermal equilibrium, and whether the same measurements were used to fit the model.

A display resolution of 0.1°Cshould not be confused with accuracy of 0.1°C. Resolution describes the displayed increment, while accuracy describes how close the result is to the true or reference temperature under defined conditions.

6.2 Recommended validation procedure

A more reliable validation should include:

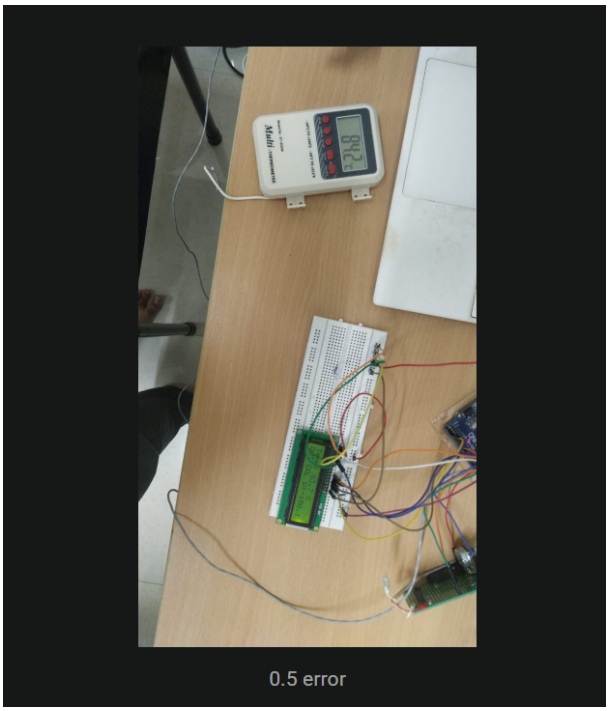
1. test several temperatures spanning the intended operating range;
2. use a reference thermometer with known uncertainty;
3. place the sensor and reference close enough to reduce temperature-gradient effects;
4. wait for thermal stability before recording measurements;

5. take repeated readings at each point;
6. calculate absolute error, MAE and RMSE where sufficient data are available;
7. identify whether each test point is inside the model's calibration domain;
8. report the reference values, measured values and test conditions.

The uncertainty of the reference thermometer must be considered before making a final accuracy claim.

Table 1: Recorded performance and verification status

Item	**Current evidence/status**
Analogue acquisition	A0 and A1 means appear in serial output
Differential feature	Calculated and printed in serial output
Least-squares calibration	Implemented in the software workflow
Serial diagnostics	Numerical intermediate values observed
Maximum observed error	Approximately 0.7°C, as reported
LCD output	Troubleshooting required; confirm final output
D8 unit toggle	Requested feature; confirm both transitions
Independent multi-point validation	Further records required
Certified accuracy specification	Not established



7 Enclosure Design and Mechanical Integration

7.1 Design objective

The enclosure is intended to protect the assembled circuit, support the LCD and boards, and provide usable access to the display and controls. It must accommodate the Arduino, perfboard, LCD, contrast potentiometer, push button, sensor lead and internal wiring without forcing the boards to bend or allowing parts to move freely when the instrument is handled.

The mechanical design is therefore part of instrument reliability, not merely an external cosmetic feature. A loose button, unsupported LCD or strained sensor wire can cause intermittent operation even when the electronic circuit is correct.

7.2 Recorded dimensions

The following measurements were recorded during the enclosure-design work:

Table 2: Recorded measurements used for enclosure planning

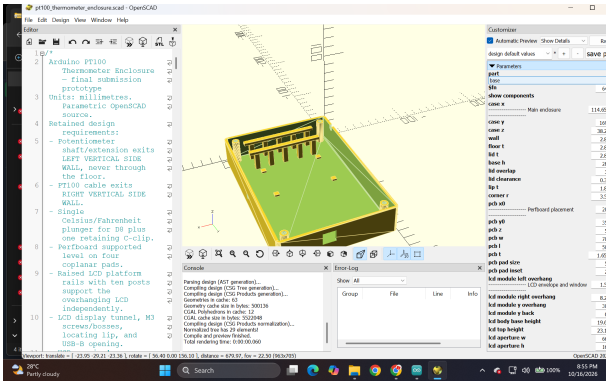
Component/feature	**Recorded measurement**
Perfboard footprint	Approximately 50 × 70 mm
Perfboard thickness	Approximately 1.6–1.7 mm
LCD module	JHD162A, 16-by-2 character LCD
LCD base height above perfboard	Approximately 19.6 mm
LCD height including display	Approximately 23.1 mm
Display protrusion	Approximately 8.2 mm

These are recorded design inputs, not universal dimensions for every LCD module. Actual components should be remeasured before final fabrication because module revisions, mounting geometry and the chosen measurement datum can affect the fit.

7.3 Raised LCD support platform

The LCD was identified as a component that should not rely entirely on its soldered pins or the perfboard for mechanical support. The enclosure design therefore includes a raised support platform intended to carry the LCD's mechanical load into the enclosure and reduce the risk of perfboard tilt.

The platform must contact the intended structural portions of the LCD without pressing on solder joints, exposed conductors or nearby components. It must leave enough underside clearance for the LCD connector and wiring. The front opening must line up with the visible display while preserving sufficient surrounding material to support the panel.



7.4 Captive push-button plunger

The push-button mechanism was required to move smoothly while remaining captive when the enclosure is turned upside down. A retaining shoulder, lip or guided plunger can prevent the button from falling out, provided that the geometry allows sufficient motion and printing clearance.

The plunger should move along a defined axis and actuate the electrical switch without binding against the enclosure. Its travel should not force the switch beyond its intended actuation point or press the PCB into the enclosure. The clearance must be checked on a printed part: too little clearance can cause binding, while too much can cause wobble or allow the plunger to escape.

7.5 Potentiometer and PT100 lead opening

The contrast potentiometer needs an aligned opening for its shaft or knob. The opening must allow the control to be adjusted without the shaft rubbing against the enclosure wall or the component body being loaded sideways.

A dedicated opening is also needed for the PT100 lead. The cable should not be bent sharply at the enclosure edge or pull directly on soldered connections. Where feasible, strain relief should transfer cable loads to the enclosure. The opening should be large enough for the cable and its movement but not unnecessarily large.

7.6 Internal mounting and protection

The Arduino and perfboard should be held securely enough that the boards cannot shift during normal handling. Mounting points must avoid exposed solder joints and should not put excessive stress on the PCB. Wire routing should avoid sharp edges, moving parts and the button plunger's path.

The assembled enclosure should be checked in multiple orientations. Turning the case over should not allow the LCD, board or internal wires to fall against other components. Any supports or fasteners should be checked for possible contact with exposed electrical connections.

7.7 Printability and structural checks

The enclosure is intended to be 3D printed. Its strength and fit depend on wall thickness, corner geometry, mounting-post dimensions, print orientation and material. The design should be checked for thin walls near openings, fragile retaining features, insufficient clearances and interference between the lid and internal parts.

Before accepting the enclosure as complete, the following checks should be performed:

- the Arduino and perfboard fit without being forced or bent;
- the raised LCD support carries the intended load;
- the button moves freely and remains captive when inverted;
- the potentiometer and sensor cable can be accessed as intended;
- screw bosses and retaining features have adequate material around them;
- the lid can close without pinching wires;
- no enclosure part can contact exposed conductive leads;

- the assembly can be opened for debugging and maintenance;
- the printed prototype has no cracks, severe warping or unintended interference.

A CAD rendering alone cannot prove structural integrity. Final acceptance requires a fit check with the actual electronics and a physical inspection of the printed part.

7.8 Ventilation and sensor placement

Any ventilation openings should preserve adequate structural material and avoid exposing the circuit unnecessarily. The sensor should be positioned so the measurement is not unduly affected by heat from the Arduino or other internal components. Thermal isolation and the placement of the sensor lead should be considered together with enclosure ventilation.

8 System-Level Testing Checklist

The final instrument combines the sensor and analogue circuit, the Arduino calculation, the LCD and controls, and the enclosure. Each subsystem can work in isolation while a problem appears during assembly, so the checks below should be performed on the fully assembled instrument.

Table 3: Final verification checklist

Test	**Status**
Arduino powers up reliably	To confirm
Analogue readings respond to temperature changes	To confirm
Repeated readings are sufficiently stable	To confirm
Calibration coefficients match the intended dataset	To verify
Calibration-domain limits are correct	To verify
Serial output agrees with the calculation	Partly observed
LCD displays temperature and status correctly	To confirm
LCD text fits within 16 characters per line	To confirm
D8 switches Celsius to Fahrenheit	To confirm
Second press returns Fahrenheit to Celsius	To confirm
Holding the button does not cause repeated toggles	To confirm
Error is documented against reference readings	Partial result reported
Arduino and perfboard are securely mounted	To confirm
LCD support transfers load to the enclosure	To confirm
Button remains captive when the enclosure is inverted	To confirm
PT100 lead has suitable strain relief	To confirm
No enclosure feature contacts exposed leads	To confirm
Enclosure can be opened for maintenance	To confirm

The status column intentionally distinguishes evidence already seen in the recorded output from tests that still need confirmation. Update it after carrying out the corresponding physical test; do not mark an item as passed merely because the design was intended to support it.

9 Challenges and Lessons Learned

9.1 A plausible result may still be outside calibration

The serial output showed that a numerical temperature can be calculated while the status remains `OUT_OF_CAL_ESTIMATE`. This demonstrates why the model's input domain and its numerical output must be checked separately. An extrapolated value should not be presented as having the same validation status as an in-range reading.

9.2 Debug the signal chain in stages

Troubleshooting is more efficient when the raw or averaged ADC counts, derived feature, fitted temperature, formatted string and LCD output are verified separately. This prevents a display error from being mistaken for a calibration error and helps identify faults in the correct part of the system.

9.3 Accuracy and display resolution are different

A display that shows one decimal place does not imply that the thermometer is accurate to one tenth of a degree. The reported maximum observed error of about 0.7°Cis useful preliminary evidence, but a stronger specification needs documented reference comparisons and a defined temperature range.

9.4 Mechanical design affects electrical reliability

Supporting the LCD, securing the boards, retaining the push button and providing cable strain relief all reduce the chance of mechanical loads damaging electrical connections. These requirements must be checked on the assembled, printed enclosure rather than assumed from the CAD model.

10 Conclusion and Remaining Work

The project progressed from assembling a PT100 measurement setup to developing a least-squares calibration workflow, implementing the calculation on an Arduino, and using serial diagnostics to inspect the measurement process. The recorded serial example shows averaged ADC readings, a differential feature, a fitted temperature and an explicit out-of-calibration status. A maximum observed error of approximately 0.7°Cwas reported during testing.

The LCD presentation required troubleshooting, and the calibration status highlighted the need to distinguish a numerical estimate from a validated measurement. The Celsius/Fahrenheit toggle on D8 and the enclosure's mechanical performance should be described as complete only after they have been confirmed on the assembled prototype.

The remaining priorities are to verify the final LCD wiring and formatting, test both directions of the unit toggle, check that the calibration coefficients and limits match the dataset, record a small independent validation table, and physically check the enclosure's fit, button retention, LCD support and cable routing.

Final record to complete after testing

Record	Final value/evidence
Date of final test	
Reference thermometer used	
Test temperature range	
Maximum absolute error	
LCD test result	
D8 button test result	
Enclosure fit and retention test	

Photograph Notes

The image commands use descriptive placeholder filenames, for example `thermometer_overview.jpg`, `circuit_assembly.jpg` and `serial_monitor.jpg`. These names have not been verified against the GitHub repository. To insert the real photographs, copy them into an `images` subfolder next to this source file and update the filenames in the corresponding
`projectimage` commands. If a file is absent, the source prints a labelled placeholder rather than failing to compile.